



MLOps Lifecycle

Introduction to MLOps

MLOps (Machine Learning Operations) is a set of practices that automates and manages the end-to-end machine learning lifecycle; from data preparation to model deployment and ongoing monitoring. The term combines "Machine Learning" with "Operations," reflecting its roots in the DevOps philosophy of bridging development and deployment workflows.

At its core, MLOps aims to:

- Standardise and automate ML workflows
- Ensure reproducibility across experiments and environments
- Enable collaboration between data scientists, ML engineers, and operations teams
- Maintain model reliability and performance after deployment

MLOps vs. DevOps

While MLOps borrows heavily from DevOps principles, it introduces unique challenges due to the data-driven and experimental nature of machine learning.

Dimension	DevOps	MLOps
Primary focus	Code deployment	Code, data, and model management
Lifecycle scope	Application lifecycle	ML lifecycle
Model retraining	Not applicable	Continuous model retraining
Monitoring scope	System/service metrics	Data quality and model performance
Key concerns	Software reliability	Data drift, experiment tracking, model governance

MLOps deals with challenges that DevOps does not face: **data drift handling**, **model retraining**, and **experiment tracking**. These require additional tooling, processes, and governance practices.

Traditional ML vs. MLOps

Many organisations start with a "Traditional ML" approach — a model is trained once, deployed manually, and rarely revisited. MLOps transforms this into a continuous, automated lifecycle.

Dimension	Traditional ML	MLOps
Model lifecycle	One-time model	Continuous lifecycle
Training process	Manual training	Automated pipelines
Monitoring	No systematic monitoring	Continuous monitoring
Model updates	Static models	Continuous retraining

The transition from Traditional ML to MLOps is not just a tooling change — it represents a **cultural and organisational shift** in how teams think about model development and maintenance.

The Problems MLOps Solves

Without MLOps, organisations encounter a predictable set of failures as they try to scale machine learning from experiments to production systems.

- **Lack of Reproducibility:** Data scientists frequently struggle to reproduce model results. Without versioned data, tracked experiments, and documented environments, reproducing a model becomes nearly impossible; especially when team members change or time passes.
- **Manual Deployment Processes:** Deploying ML models manually is slow, error-prone, and inconsistent. It creates bottlenecks between data science teams and engineering/infrastructure teams, slowing down the time-to-value for machine learning investments.
- **Model Performance Degradation:** Models trained on historical data degrade over time as real-world patterns shift. Without monitoring, this degradation goes unnoticed until it causes significant business impact. For example, a fraud detection model trained on pre-pandemic spending patterns will fail as consumer behavior changes.



- **Poor Collaboration:** In many organisations, data scientists, ML engineers, and operations teams work in silos, using different tools and conventions. MLOps provides shared infrastructure, workflows, and conventions that enable seamless collaboration.

Business Value of MLOps

Adopting MLOps delivers measurable business outcomes:

- **Faster time-to-production:** Automated pipelines reduce model deployment time from weeks to hours
- **Reduced operational risk:** Systematic monitoring and validation prevent silent model failures
- **Improved model quality:** Continuous retraining ensures models remain accurate as data evolves
- **Cost efficiency:** Automation reduces the manual effort required to maintain production models

The MLOps Lifecycle

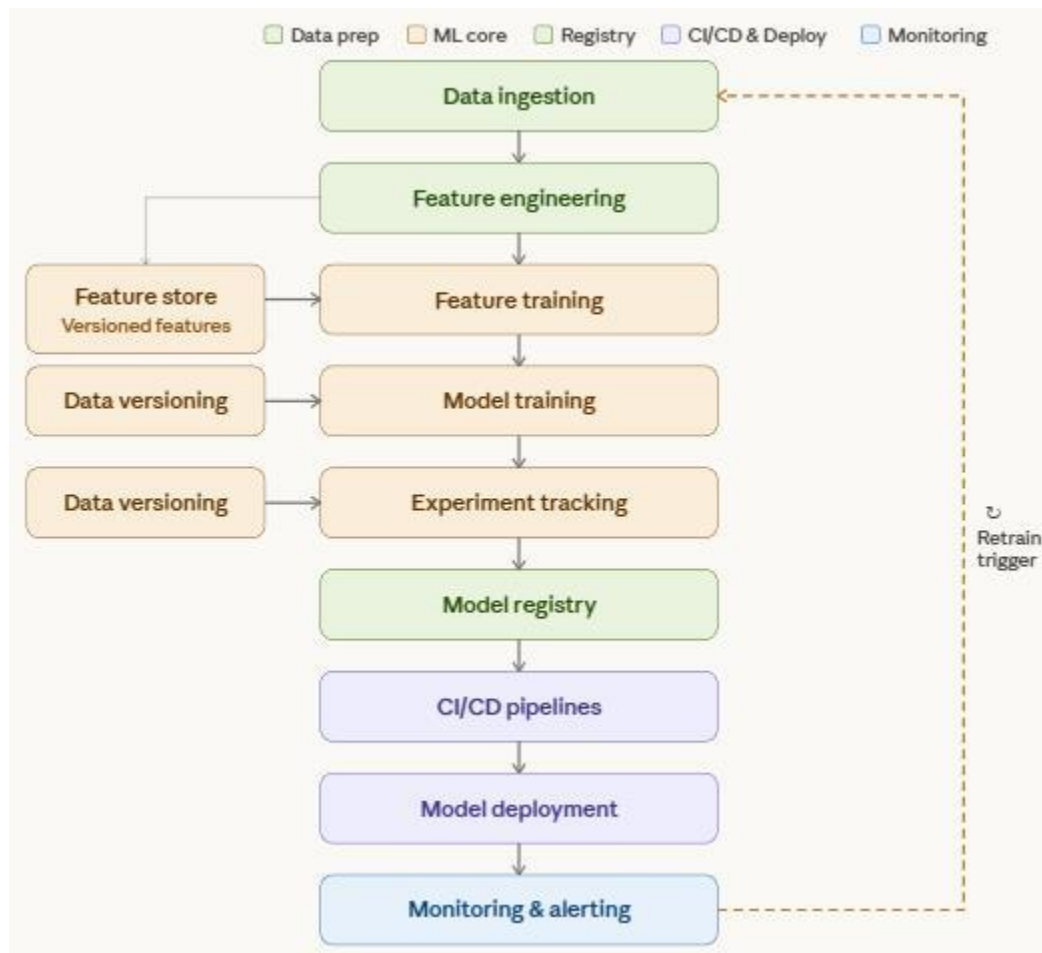
The MLOps lifecycle is a closed-loop system. Unlike traditional software, it never truly "finishes" — models must be continuously evaluated, retrained, and redeployed to maintain their value.

The key stages of the MLOps lifecycle are:

1. **Data Engineering and Feature Management**
2. **Experiment Tracking and Model Training**
3. **CI/CD and Model Registry**
4. **Model Deployment**
5. **Model Monitoring**
6. **Retraining and Continuous Improvement**

MLOps Architecture

The MLOps architecture connects all lifecycle stages through automated pipelines:



Each stage feeds into the next, and monitoring outputs feed back into the retraining stage, creating a continuous improvement loop.

Data Engineering and Feature Management

The Foundation of Every Model



A model is only as good as its training data. Inconsistent, unversioned, or undocumented data causes most silent ML failures. Data engineering in MLOps treats data as a **first-class engineering artefact** — subject to the same rigor as code.

Key Practices

1. **Data Versioning:** Just as code is version-controlled with Git, datasets must also be versioned. Data versioning allows teams to:

- Track which version of the data produced which model
- Roll back to a known-good dataset if quality issues are discovered
- Reproduce any past experiment with the exact data used at the time

Tools like **DVC (Data Version Control)** and **Delta Lake** enable Git-like versioning for datasets.

2. **Schema Validation:** Data pipelines should enforce schema contracts at every boundary. Schema validation ensures that incoming data matches expected types, ranges, and formats before it reaches the model. Unexpected schema changes — such as a new category in a categorical variable or a sudden increase in null values — can silently degrade model performance.
3. **Feature Stores:** A **feature store** is a centralised repository for computing, storing, and serving features to ML models. Feature stores solve a critical problem: **train/serve skew**, where features computed during training differ from those computed during serving. By centralising feature computation, feature stores ensure consistency and eliminate this discrepancy.

Popular feature stores include **Feast**, **Tecton**, and managed offerings from cloud providers.

4. **Lineage Tracking:** Data lineage tracks the origin and transformation history of every data point from source to model. This is critical for debugging model failures, ensuring compliance, and understanding the downstream impact of upstream data changes.
5. **Experiment Tracking:** Machine learning development is inherently experimental. Data scientists test dozens of configurations — different algorithms, hyperparameters, feature



sets, and preprocessing steps — before arriving at a production-ready model. Without systematic tracking, this process becomes disorganised and non-reproducible.

What Experiment Tracking Covers

Experiment tracking systems record and organise:

- **Parameters:** Hyperparameters, feature sets, data versions, and configuration settings used in each run
- **Metrics:** Accuracy, precision, recall, F1-score, loss curves, and other evaluation metrics
- **Artifacts:** Trained model files, confusion matrices, evaluation reports, and intermediate datasets
- **Code versions:** The exact code commit used to produce each experiment result

Key Benefits

- **Reproducibility:** Any experiment can be re-run with identical configuration and data
- **Model comparison:** Teams can compare multiple model runs side-by-side to select the best candidate for production
- **Knowledge sharing:** Experiment history is shared across the team, preventing redundant work and enabling collaborative improvement
- **Auditability:** A complete record of all experiments supports regulatory compliance and internal governance

Tools

Leading tools for experiment tracking include **MLflow Tracking**, **Weights & Biases (W&B)**, and **Neptune.ai**. Most MLOps platforms also provide integrated experiment tracking.



CI/CD in MLOps

Continuous Integration and Continuous Deployment

CI/CD (Continuous Integration / Continuous Deployment) is the practice of automating the build, test, and release process for software. In MLOps, CI/CD extends to ML systems — automating not just code deployments but also model validation, retraining, and production releases.

Continuous Integration (CI)

CI in MLOps encompasses:

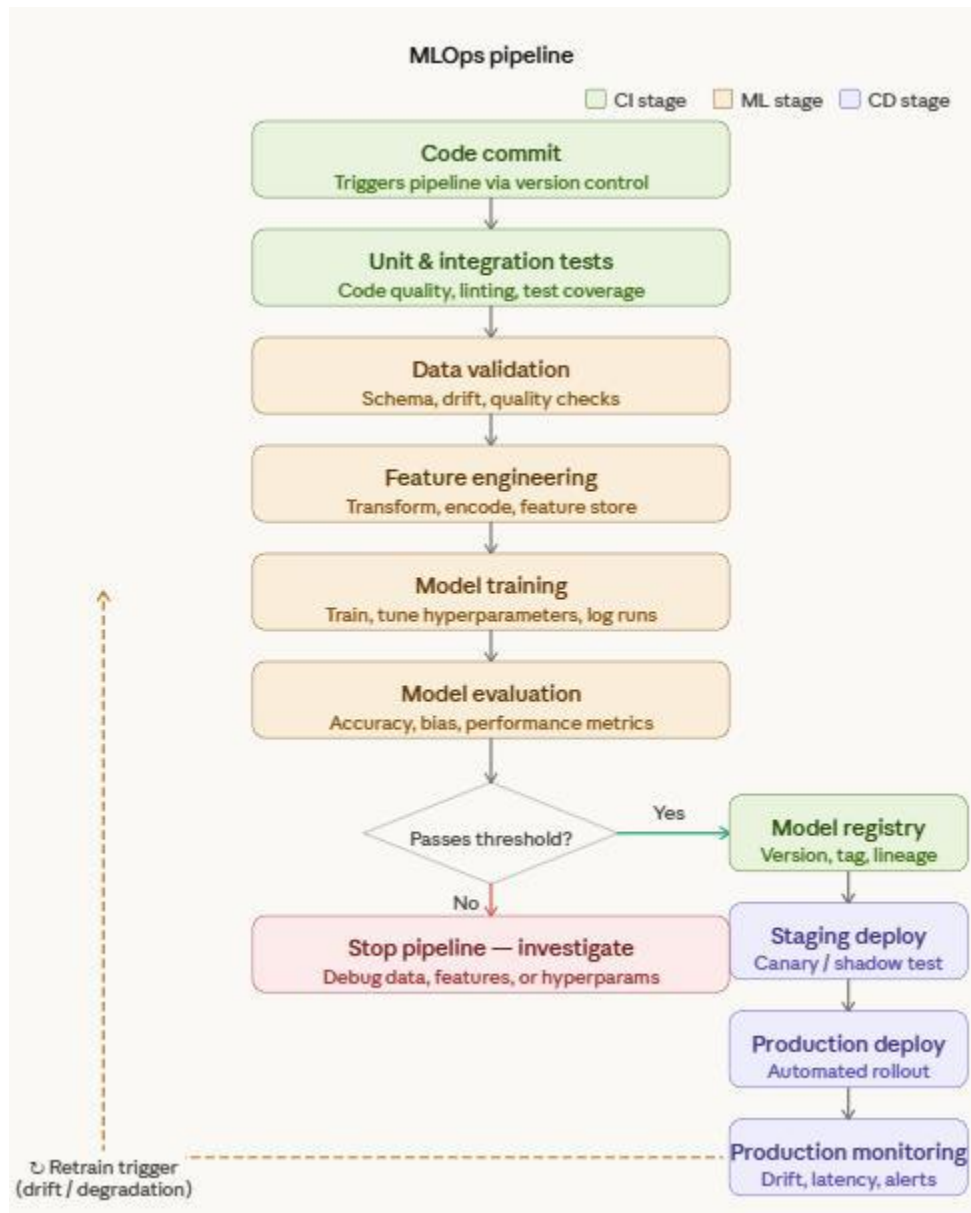
- **Code and data validation:** Automatically checking new code and data against defined quality standards
- **Automated pipeline testing:** Running unit tests, integration tests, and schema validation checks on every code commit
- **Consistency enforcement:** Ensuring that model training is reproducible across environments and team members

Continuous Deployment (CD)

CD in MLOps includes:

- **Automated model release:** Deploying validated models to production without manual intervention
- **Staged rollout:** Gradually increasing traffic to new models while monitoring performance
- **Reduced manual intervention:** Replacing error-prone manual deployment steps with reliable automation

CI/CD Pipeline Example



Model Registry

A **model registry** is a centralised repository for storing, versioning, and managing trained ML models. It serves as the "single source of truth" for all models across their development, staging, and production stages.



Key Practices

- **Model Versioning:** Every trained model is stored with metadata including the training data version, hyperparameters, evaluation metrics, and the code used to produce it. Version tracking ensures that any historical model can be retrieved, evaluated, and compared.

Stage Management

Models progress through defined stages:

- **Development:** Initial model candidates being evaluated
- **Staging:** Models that have passed quality gates and are being validated in a pre-production environment
- **Production:** Models actively serving predictions to end users
- **Archived:** Deprecated models retained for historical reference
- **Access Control:** The model registry enforces role-based access control (RBAC) to ensure that only authorised users can promote models to production. This prevents accidental deployments and supports governance requirements.
- **Lineage Tracking:** The registry records which datasets, feature versions, and experiment runs produced each model. This full traceability supports root-cause analysis when a model fails in production.
- **Tools:** **MLflow Model Registry** is the most widely used open-source option. Cloud-managed registries include **Amazon SageMaker Model Registry**, **Google Vertex AI Model Registry**, and **Azure ML Model Registry**.

Model Deployment



Deployment Approaches

1. Batch Inference

In batch inference, the model processes a large dataset at scheduled intervals (hourly, daily, weekly) and stores the predictions for downstream consumption. This approach is appropriate when:

- Predictions are not time-critical
- Large volumes of data need to be processed
- Results can be precomputed and cached

Example: Sales forecasting — predictions for next week's demand are generated every Sunday night and fed into inventory management systems.

2. Real-Time API (Online Inference)

In real-time inference, the model is exposed as a REST API (commonly built with **FastAPI** or **Flask**) and returns predictions in milliseconds in response to individual requests. This approach is appropriate when:

- Low-latency predictions are required
- Requests arrive one at a time or in small batches
- User-facing applications need immediate responses

Example: Fraud detection — a transaction is evaluated in real time as it occurs, and a risk score is returned to the payment system within 100ms.

3. Streaming Inference

Streaming inference processes a continuous stream of events in near-real time, typically using tools like **Apache Kafka** or **AWS Kinesis**. This approach bridges batch and real-time inference, supporting use cases where data arrives as a continuous stream.

Deployment Infrastructure

Approach	Latency	Volume	Example
Batch inference	High (scheduled)	Very high	Sales forecasting
Real-time API	Very low (ms)	Medium	Fraud detection
Streaming	Low (near-real-time)	High	Ad click prediction

Model Monitoring

Why Monitoring Is Critical

Deploying a model is not the final step — it is the beginning of a new phase of continuous oversight. In production, models face real-world data that may differ from their training distribution. Without monitoring, performance degradation is invisible until it causes significant harm.

What to Monitor

1. Model Performance Metrics

- Accuracy, precision, recall, F1-score, AUC-ROC
- Error rates (false positives, false negatives)
- Business KPIs tied to model output (e.g., conversion rate, churn reduction)

2. Latency and Infrastructure Metrics

- Prediction latency (time from request to response)
- Throughput (predictions per second)
- Resource utilization (CPU, GPU, memory)

3. Data Drift: Data drift occurs when the statistical distribution of the model's input data changes over time. This is one of the most common causes of model performance degradation in production.



Example: A model trained on pre-pandemic customer spending patterns will experience data drift as consumer behavior changes after the pandemic.

Detection methods include:

- **Population Stability Index (PSI):** Measures the shift in feature distributions
 - **Kolmogorov-Smirnov (KS) test:** Statistical test for distribution differences
 - **Chi-squared test:** For categorical feature distributions
4. **Concept Drift:** Concept drift occurs when the underlying relationship between input features and the target variable changes. Unlike data drift (which affects inputs), concept drift affects the fundamental mapping the model learned.

Example: Fraud patterns evolve over time as attackers adapt to detection methods. A fraud model must relearn the updated relationship between transaction features and fraud labels.

Monitoring Architecture

An effective monitoring system should include:

1. **Real-time metric collection:** Log predictions, inputs, and ground truth as they become available
2. **Drift detection algorithms:** Automatically detect statistically significant distributional changes
3. **Alerting and dashboards:** Notify teams via Slack, PagerDuty, or email when thresholds are crossed
4. **Automated retraining triggers:** Initiate retraining pipelines when drift is confirmed

Data and Model Versioning

The Importance of Versioning in ML



Unlike traditional software, where code alone determines behavior, ML model behavior is determined by code, data, and hyperparameters together. Versioning all three is essential for reproducibility, auditability, and safe rollbacks.

What to Track

Model Versions: Each trained model is assigned a unique version identifier and stored with:

- Training data version
- Hyperparameter configuration
- Evaluation metrics
- Code commit hash

Data Versions: Datasets used for training and evaluation are versioned independently. This allows any model to be retrained on its exact original data, even months after the fact.

Experiment Runs: Each training experiment is logged as a distinct run with full configuration and outcome data.

Benefits of Systematic Versioning

- **Reproducibility:** Any past model can be reconstructed from its version record
- **Comparison:** Different model versions can be evaluated side-by-side
- **Rollback:** If a new model degrades in production, the previous version can be immediately restored
- **Compliance:** Version history provides a complete audit trail for regulatory requirements

Retraining and Continuous Improvement

- **Closing the ML Loop:** Model retraining is how a production ML system adapts to change. As data distributions shift and business contexts evolve, models must be retrained on fresh data to maintain their accuracy and relevance.
- **Retraining Triggers**
 - **Schedule-Based Retraining:** The model is retrained at fixed intervals (e.g., weekly, monthly) regardless of measured performance. This approach is simple to implement and provides a baseline level of freshness. However, it may retrain unnecessarily when the model is still performing well, or fail to retrain quickly enough when performance degrades suddenly.
 - **Performance-Based Retraining:** Retraining is triggered when monitored metrics (accuracy, F1-score, business KPIs) fall below a defined threshold. This approach is more efficient than schedule-based retraining because it responds to actual need.
 - **Data-Based Retraining:** Retraining is triggered when statistical drift detection identifies a statistically significant change in the input data distribution. This is proactive — it responds to leading indicators of future degradation before metrics decline.
 - **Event-Based Retraining:** Retraining is triggered by external events that are known to shift data distributions — for example, a major regulatory change, a market disruption, or a pandemic. This requires human judgment to identify triggering events and initiate retraining accordingly.
 - **The Feedback Loop:** Effective retraining depends on a **feedback loop** that collects ground truth labels from production. Without true labels, the model can only be retrained on distribution, not on correctness.

In many real-world systems, labels are delayed or expensive to collect. Best practices include:

- Designing systems to collect implicit feedback (e.g., clicks, conversions)
- Using active learning to prioritize labeling high-uncertainty predictions
- Establishing human review pipelines for critical predictions



MLOps Tools and Platforms

Full MLOps platforms provide integrated lifecycle management across development, deployment, and monitoring.

Amazon SageMaker: SageMaker is AWS's fully managed ML platform. It provides integrated tools for data labeling, training, hyperparameter tuning, model hosting, and monitoring. SageMaker Pipelines enables end-to-end ML workflow automation, and SageMaker Model Monitor automatically detects data drift in production.

Google Cloud Vertex AI: Vertex AI is Google Cloud's unified ML platform, integrating AutoML and custom model training. It offers Vertex AI Pipelines for workflow orchestration, the Vertex Feature Store for centralized feature management, and integrated monitoring and explainability tools.

Azure Machine Learning: Azure ML provides enterprise-grade MLOps capabilities including experiment tracking, the Azure ML Model Registry, automated ML pipelines, and integration with Azure DevOps for CI/CD workflows.

Tools by MLOps Stage

Stage	Popular Tools
Experiment Tracking	MLflow , Weights & Biases (W&B) , Neptune.ai
Data and Model Versioning	DVC (Data Version Control) , Delta Lake , LakeFS
Model Registry	MLflow Model Registry , SageMaker Model Registry
Pipeline / Workflow	Apache Airflow , Kubeflow Pipelines , ZenML
Deployment	FastAPI , BentoML , Seldon Core , KServe
Orchestration	Kubernetes , Kubeflow , Metaflow
Monitoring	Evidently AI , WhyLabs , Arize AI , Prometheus + Grafana

Key Challenges in MLOps



Understanding MLOps Challenges

Implementing MLOps at scale is not straightforward. Organizations face both technical and organizational challenges that require deliberate strategies to overcome.

Challenge	Description
Model Drift	Gradual degradation in model performance over time as real-world data distributions shift
Data Inconsistencies	Variations in data quality and distribution across environments and time periods
Reproducibility Issues	Difficulty in replicating experiments and results across team members and environments
Deployment Delays	Slow and complex model release processes caused by manual steps and inadequate tooling
Lack of Collaboration	Gaps and friction between data science, engineering, and operations teams

Strategies for Overcoming Challenges

- **For model drift:** Implement robust monitoring with automated drift detection and retraining triggers
- **For data inconsistencies:** Enforce schema validation and use feature stores to standardize feature computation
- **For reproducibility:** Version all code, data, and models; use container-based environments (Docker, Kubernetes)
- **For deployment delays:** Adopt CI/CD pipelines and automated model validation gates
- **For collaboration gaps:** Establish shared tooling, common interfaces, and cross-functional MLOps teams

Benefits of MLOps

- **Faster Deployment:** MLOps enables faster and more efficient deployment of models into production environments. Automated pipelines eliminate manual handoffs and reduce deployment time from weeks to hours.



- **Better Collaboration:** Shared pipelines, version-controlled assets, and standardized tooling improve collaboration between data scientists, engineers, and operations teams. Teams can work concurrently on different stages of the ML lifecycle without stepping on each other's work.
- **Reproducibility:** MLOps ensures that experiments, data, and models can be consistently reproduced across environments and team members. This is essential for scientific rigor, debugging, and regulatory compliance.
- **Scalability:** MLOps provides the infrastructure and automation needed to scale machine learning operations — handling more models, more data, and more frequent deployments without proportionally increasing headcount.
- **Reduced Risk:** Systematic testing, monitoring, and rollback capabilities minimize errors, reduce production failures, and improve the overall reliability of ML systems.

MLOps Adoption Maturity

Levels of MLOps Maturity

Organizations do not adopt MLOps overnight. They typically evolve through three maturity levels:

- **Level 1 — Manual Process**
Model development, deployment, and monitoring are handled manually with minimal automation. Data scientists train models in notebooks, hand off artifacts to engineers, and models are deployed through ad hoc scripts. There is no systematic monitoring or versioning.
- **Level 2 — Partial Automation**
Some stages (training, testing, or deployment) are automated, but workflows are not fully integrated. Teams begin adopting tools for experiment tracking and model versioning, but end-to-end pipelines are still fragmented.
- **Level 3 — Fully Automated Pipelines**
End-to-end pipelines are automated with continuous integration, deployment, and monitoring. Models are automatically retrained when drift is detected, validated against quality gates, and deployed to production without manual intervention.



MLOps in Practice — Industry Case Studies

- **Netflix:** Netflix manages thousands of ML models powering recommendations for millions of subscribers. The company built **Metaflow** (now open-source) to provide consistent ML workflows across all teams. Netflix uses **Statistical Model Drift Detection (SMDD)** to flag anomalies in real time. The result is rapid, safe model deployment with minimal downtime.
- **Uber:** Uber built an internal MLOps platform called **Michelangelo** to standardize model training, deployment, and monitoring across all teams. Michelangelo enables hundreds of data scientists to deploy models to production in hours rather than months.
- **Airbnb:** Airbnb faced severe reproducibility issues across teams. By implementing MLOps practices — including versioned feature pipelines and systematic experiment tracking — Airbnb achieved consistent model behavior across environments and dramatically reduced debugging time.
- **Amazon:** Amazon uses MLOps to monitor models in production at massive scale across AWS SageMaker. Automated retraining triggers are activated when model performance degrades, ensuring that models stay accurate as data distributions shift and directly protecting revenue.

Application: Telecom Churn Prediction

A representative MLOps application in the telecom industry involves predicting customer churn:

1. **Data ingestion:** Customer demographics and behavioral data are collected and versioned
2. **Model training:** A churn prediction model is trained on historical data
3. **Deployment:** The model is deployed as a production API
4. **Monitoring:** Churn patterns are monitored in production continuously
5. **Drift detection:** If drift is detected in customer behavior data, retraining is automatically triggered
6. **Retraining:** The model is retrained on fresh data and redeployed

This closed-loop system ensures the churn model remains accurate even as customer behavior evolves.



Common Mistakes in MLOps

- **Ignoring Data Versioning:** Not tracking data changes leads to inconsistent and non-reproducible results. If the training data for a production model is unknown, debugging failures becomes nearly impossible.
- **No Production Monitoring:** Failing to monitor models in production results in unnoticed performance degradation. Silent model failures can cause significant business harm before anyone notices the root cause.
- **Deploying Without Testing:** Skipping validation and testing before deployment can lead to unreliable and incorrect predictions reaching end users. Every model should pass a defined set of quality gates before promotion to production.
- **Treating ML Like Traditional Software:** ML systems have unique dependencies on data and model behavior that traditional software engineering practices do not account for. Teams that ignore these differences find ML systems much harder to maintain, debug, and improve.

Key Takeaways

- **MLOps is essential** for deploying and managing machine learning systems in production at scale
- **The full lifecycle matters**, not just model development — data, training, deployment, monitoring, and retraining are equally important
- **MLOps bridges the gap** between ML experimentation and real-world production applications
- **Automation and CI/CD** ensure reliability, consistency, and speed in model delivery
- **Continuous feedback loops** help maintain and improve model performance over time
- **Cross-functional collaboration** across data science, engineering, and operations teams is critical for MLOps success

Project Overview



This textbook explains the Python project as a complete MLOps case study for e-commerce purchase prediction. It is built around a production-style workflow that begins with a weak baseline, moves through rigorous model selection, deploys a full pipeline, detects drift, retrains when needed, and serves predictions through both batch and API interfaces.

The project is not framed as only a machine learning exercise. It is presented as a reliability and systems problem where model quality, reproducibility, monitoring, deployment gating, and retraining all matter together.

Problem Statement

The business problem is to predict whether an e-commerce browsing session will end in a purchase so that the business can identify high-intent sessions and make better marketing or offer decisions. The ML task is a binary classification problem where the target is Revenue, with 1 meaning purchase and 0 meaning no purchase.

The project uses one row per browsing session with features such as page counts, durations, bounce rate, exit rate, PageValues, Month, VisitorType, SpecialDay, and Weekend. The dataset is highly imbalanced, with only about 12–15% of sessions ending in purchase, which makes plain accuracy misleading for evaluation.

A second challenge is temporal shift. The model is trained on February to October traffic, while November to December traffic arrives after deployment and behaves differently because of the holiday season.

Business Context and Why Accuracy Fails

The notebook files repeatedly stress that false negatives are far more expensive than false positives. In `model_selection.py`, the business cost is described as about \$85 for missing a buyer and about \$3 for unnecessarily targeting a non-buyer, giving a cost ratio of 28:1.

That is why this project uses F1 score on the minority class as the primary metric instead of accuracy. The notebook also states that a model predicting only “no purchase” can still reach around 87–88% accuracy, while being useless for the actual business objective.



Key evaluation logic

- Primary metric: F1 on buyers (Revenue = 1).
- Secondary metrics: AUC-ROC, AUC-PR, precision, and recall.
- Deployment gates: F1, recall, AUC-PR, and AUC-ROC must cross thresholds before promotion.
- Class imbalance handling: $\text{scale_pos_weight} = \text{n_negative} / \text{n_positive}$.

Dataset and Learning Scenario

The notebook identifies the problem as based on the UCI Online Shoppers Purchasing Intention dataset, with 12,330 sessions and 18 features. The notebook flow narrows this into a production simulation where 7,317 sessions from February to October are available at training time, and 4,725 November–December sessions arrive later as post-deployment data.

This timeline matters because the project intentionally separates model building from later operational monitoring. The current batch is not allowed into initial training, which makes the evaluation and later drift handling more realistic.

Notebook Flow

The learning sequence is explicit and should be followed in order. The recommended run order is shown below.

Must run in order

1. `python baseline.py`
2. `python model_selection.py`
3. `python main.py`
4. `python retrain.py --dry-run`

5. `python retrain.py --mixed`
6. `uvicorn serve:app --reload --port 8000`
7. `python batch_infer.py`
8. `python plot_logs.py`

File-by-File Interpretation

File	What it does
<code>baseline.py</code>	First-pass analysis — demonstrates common pitfalls
<code>model_selection.py</code>	Compares multiple models and selects the best one
<code>main.py</code>	Entry point — runs the full production pipeline
<code>retrain.py</code>	Handles drift-triggered retraining strategies
<code>serve.py</code>	Provides FastAPI endpoint for real-time predictions
<code>batch_infer.py</code>	Performs batch scoring on entire dataset
<code>plot_logs.py</code>	Visualizes logs (metrics, drift, latency, registry)
<code>src/config/config.py</code>	Stores configuration (thresholds, features, constants)
<code>src/data/loader.py</code>	Loads dataset (UCI or fallback data)
<code>src/data/prepare.py</code>	Splits data into train and current (temporal split)
<code>src/data/validate.py</code>	Validates schema and data quality
<code>src/features/build_features.py</code>	Handles feature engineering (encoding, imputation)

File	What it does
src/models/train.py	Trains model with imbalance handling
src/models/evaluate.py	Evaluates model and applies deployment checks
src/pipeline/run_pipeline.py	Orchestrates full pipeline execution
src/monitoring/drift.py	Detects data drift across features
src/tracking/experiment.py	Logs experiment results
src/registry/versioning.py	Manages model versions and lifecycle
src/utils/lineage.py	Tracks data, model, and code relationships

Output Files

File	Contents
data/train.csv	Training data (Feb–Oct sessions)
data/current.csv	Post-deployment data (Nov–Dec sessions)
models/feature_pipeline.json	Saved preprocessing details (medians, schema) for inference
models/registry.json	Model registry with lifecycle states
models/*.pkl	Versioned trained model files
logs/experiments.csv	History of runs with parameters and metrics

File	Contents
logs/drift.csv	Records of drift detection across runs
logs/latency.csv	Inference time tracking
logs/lineage.json	Snapshot of model-data-code relationshi

Supporting components of the MLOps system

Following are the components included for understanding , not executed directly.

`src/*` modules

- Internal building blocks of the pipeline (validation, features, training, etc.)
- Executed indirectly via the main pipeline (`main.py`)
- Purpose: understand system design and modular structure

`serve.py` (before API run)

- Defines API logic (schemas, prediction flow)
- Not run using `python serve.py`
- Executed only via an API server (e.g., `uvicorn serve:app`)
- Purpose: demonstrate deployment setup before actual serving

The notebook structures the lecture in phases: problem context, failure case, model selection, pipeline entry, pipeline backbone, deep dive into components, full pipeline run, monitoring, retraining, batch inference, observability, and API serving.

Phase 1: baseline.py — Failure Case

`baseline.py` is designed to show code that runs successfully but is still unsafe for production. Its central lesson is that “code that runs” is not the same as “code that works” in an MLOps setting.

Main pitfalls demonstrated

Pitfall	What the code does	Why it is wrong
Hardcoded paths	Uses <code>pd.read_csv("data/train.csv")</code>	Breaks portability and robustness.
Wrong encoding	Uses <code>LabelEncoder()</code> on <code>Month</code> , <code>VisitorType</code> , and <code>Weekend</code>	Creates false ordinal meaning for nominal categories.
Wrong model sweep metric	Sorts models by accuracy	Hides weak buyer detection under class imbalance.
Weak split practice	Uses <code>train_test_split(X, y, test_size=0.20)</code> without stratification	Can distort minority-class representation.
No class weighting	Trains <code>RandomForestClassifier(n_estimators=100)</code> without imbalance handling	Biases predictions toward the majority class.
No saved pipeline	Does not save encoders, medians, or feature order	Causes training-serving mismatch.
No drift monitoring	Ignores post-deployment distribution changes	Performance can degrade silently.

Code syntax from baseline.py

This syntax is intentionally shown as a bad practice in this project because `Month` and `VisitorType` are nominal categories, not ordinal ones.

```
le_month = LabelEncoder()
le_visitor = LabelEncoder()
le_weekend = LabelEncoder()
```



```
df_feat["Month"] = le_month.fit_transform(df_feat["Month"].astype(str))
df_feat["VisitorType"] = le_visitor.fit_transform(df_feat["VisitorType"].astype(str))
df_feat["Weekend"] = le_weekend.fit_transform(df_feat["Weekend"].astype(str))
```

Another syntax

The baseline file uses this to show how an apparently standard workflow can still be misleading when stratification, cost-sensitive learning, feature persistence, and operational controls are missing.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)

model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
preds_test = model.predict(X_test)
```

Phase 2: model_selection.py — Rigorous Comparison

model_selection.py corrects the flaws in the baseline and acts as the audit trail for selecting the production model. It compares Logistic Regression, Random Forest, XGBoost, and LightGBM under the same split, the same preprocessing, and the same imbalance-aware logic.

Techniques used

- Stratified train/test split with stratify=y_all.
- Feature building through the production feature pipeline.
- Scaling for Logistic Regression with StandardScaler().
- Class weighting for Logistic Regression and Random Forest.
- scale_pos_weight for XGBoost and LightGBM.
- Minority-class F1 as the primary selection metric.



Key Implementation Steps and Code Snippets

Splits the dataset into training and testing sets. Stratification ensures class imbalance is preserved in both sets.

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_all, y_all, test_size=0.20, random_state=SEED, stratify=y_all  
)
```

Scales features to have zero mean and unit variance. Fit is done on training data, and the same transformation is applied to test data.

```
scaler = StandardScaler()  
X_tr_sc = scaler.fit_transform(X_train)  
X_te_sc = scaler.transform(X_test)
```

Initializes a Logistic Regression model with class balancing. Handles imbalanced data by giving more importance to the minority class.

```
lr = LogisticRegression(class_weight="balanced", max_iter=1000, random_state=SEED)
```

Creates a Random Forest model with controlled depth and balanced class weights. Uses multiple trees in parallel to improve performance and reduce overfitting.

```
rf = RandomForestClassifier(  
    n_estimators=200, max_depth=15, class_weight="balanced",  
    random_state=SEED, n_jobs=-1  
)
```

Initializes an XGBoost model with boosting and imbalance handling. Uses `scale_pos_weight` to handle class imbalance and optimize performance.



```
xgb = XGBClassifier(  
    n_estimators=300, learning_rate=0.05, max_depth=6,  
    scale_pos_weight=scale_w,  
    random_state=SEED, n_jobs=-1,  
    eval_metric="logloss", verbosity=0,  
)
```

This explains that LightGBM is chosen because boosted trees handle the skewed, non-linear tabular patterns well, especially around strong variables such as `PageValues`.

Phase 3: `main.py` — Pipeline Entry Point

`main.py` is intentionally simple and acts as the controller for the entire system. It delegates all core orchestration to the pipeline module.

Entry-point syntax

```
from src.pipeline.run_pipeline import run_pipeline  
  
if __name__ == "__main__":  
    run_pipeline()
```

The docstring in `main.py` clearly frames the deployment timeline: train on February–October, evaluate on a hold-out split from that period, then score November–December traffic only after deployment. It also lists expected pre-deployment performance bands and connects Stage 10 to drift-triggered retraining.

Pipeline Backbone: 12 Stages

The notebooks says the heart of the system is `src/pipeline/run_pipeline.py`, explained as a 12-stage backbone. Even though that source file is not separately mentioned, the stage-by-stage logic is described in detail inside the notebooks and walkthrough document.



Stage 0 — Initialisation

The pipeline fixes the random seed for reproducibility and creates required directories to avoid runtime errors.

```
# Sets reproducibility + creates directories  
  
np.random.seed(SEED)  
  
os.makedirs([LOG_DIR, MODELS_DIR, DATA_DIR], exist_ok=True)
```

Stage 1 — Data Loading

The pipeline loads training data from February–October and current data from November–December. The notebook repeatedly emphasizes that current data is loaded for monitoring and scoring, not for initial model training.

```
train_df = pd.read_csv("data/train.csv") # Feb-Oct (7,317 rows)  
  
current_df = pd.read_csv("data/current.csv") # Nov-Dec (4,725 rows)
```

Stage 2 — Validation

The pipeline validates schema, ranges, and null handling before model processing. The design principle is “fail fast, not silently.”

```
assert train_df["Revenue"].dtype == "int64"  
  
assert train_df["PageValues"].between(0, 1000).all()
```

Stage 3 — Feature Engineering

The feature stage imputes missing values, one-hot encodes categorical columns, and aligns train/inference columns to prevent training-serving skew.

```
from sklearn.pipeline import Pipeline  
  
feature_pipe = Pipeline([  
  
    ("imputer", SimpleImputer(strategy="median")),  
  
    ("encoder", OneHotEncoder(drop="first", sparse=False))  
  
])
```



```
# Saves training medians → models/feature_pipeline.json
```

Stage 4 — Train/Test Split

Only the training-period data is split, and the split is stratified so that the minority class ratio is preserved.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_train_period, y_train_period,
    test_size=0.20, random_state=SEED, stratify=y_train_period
)
```

Stage 5 — Training

The selected LightGBM model is trained with imbalance-aware weighting.

```
model = lgb.LGBMClassifier(
    scale_pos_weight=(y_train==0).sum()/(y_train==1).sum() # ~7.1
)
```

Stage 6 — Evaluation

The model is evaluated only on the hold-out test split using F1, AUC-ROC, and AUC-PR.

```
metrics = {
    "f1": f1_score(y_test, preds),
    "auc_roc": roc_auc_score(y_test, proba[:,1]),
    "auc_pr": average_precision_score(y_test, proba[:,1])
}
```

Stage 6b — Slice Evaluation

The notebook adds slice evaluation by `VisitorType` and `Month` to identify performance differences across segments.

```
slice_results = {}
for segment in ["VisitorType", "Month"]:
```



```
slice_f1 = evaluate_by_group(X_test, y_test, segment)

slice_results[segment] = slice_f1
```

Stage 7 — Deployment Gates

Threshold-based gates decide whether the model may be promoted. The key idea is that training success does not automatically mean deployment success.

```
if (metrics["f1"] >= 0.65 and
    metrics["recall"] >= 0.60 and
    metrics["auc_pr"] >= 0.60):
    promote_to_staging()
```

Stage 8 — Experiment Log

Pipeline outputs are appended to `logs/experiments.csv` for experiment tracking.

```
experiment_df = pd.DataFrame([
    {
        "timestamp": now(), "f1": metrics["f1"],
        "model_id": model_version, "n_features": len(features)
    }
])

experiment_df.to_csv("logs/experiments.csv", mode="a")
```

Stage 9 — Registry

The trained model is saved as a versioned `.pkl` file and lifecycle state is tracked through candidate, staging, and production in `models/registry.json`.

```
save_model(model, f"models/model_{timestamp}.pkl")

registry = {"version": v, "status": "candidate→staging→production"}
```

Stage 10 — Drift Detection

Train and current distributions are compared so the system can detect holiday-season shift in important features.



```
for feat in ["PageValues", "BounceRates"]:  
    shift = ks_2samp(train_feat, current_feat).statistic  
  
    if shift > DRIFT_THRESHOLD: flagged.append(feat)
```

Stage 11 — Predictions

The current batch is scored and both predictions and latency are logged.

```
current_preds = model.predict(X_current)  
  
log_predictions(current_preds)  
  
log_latency(inference_time_ms)
```

Stage 12 — Lineage

The project records links among model, data, code, and environment for observability and reproducibility.

```
lineage = {  
    "model_file": model_path,  
    "data_files": ["train.csv", "current.csv"],  
    "git_hash": get_commit(), "python_version": sys.version  
}
```

Feature Engineering and Pipeline Consistency

One of the most important technical ideas in this project is that the same feature logic must be used during training and inference. The notebooks says this stage stores training-time transformations such as imputation values and encoding structure so they can be reused later.

Why this matters



If inference rebuilds preprocessing from the incoming batch, the model may receive features in a different order or with different imputation behavior. That creates training-serving skew, which is one of the central failure modes illustrated by the baseline file.

Batch and Serve Inference Flow

Following shows how `batch_infer.py` and `serve.py` use the *exact same feature transformations* learned during training — preventing the #1 production ML failure: training-serving skew.

`batch_infer.py` implements post-deployment batch scoring for a CSV of sessions. It contrasts batch inference with online inference and explains that true batch inference scores the entire feature matrix in one pass rather than looping through single-row API calls.

```
From batch_infer.py: X, y_true, feature_cols = build_features(df, training=False)
```

```
From serve.py: X, _, feature_cols = build_features(row_df, training=False)
```

The `training=False` flag is the critical signal that inference must reuse saved training-time transformations rather than fitting new ones.

Key techniques used

- Load production model from the registry, not just the newest file.
- Rebuild features using the saved training-time pipeline.
- Align inference columns with the exact training column set.
- Score all rows together using vectorized prediction.
- Attach prediction probability, intent band, version, and timestamp to the output CSV.
- Log latency and predictions for observability.

Production Inference Workflow and Key Code Snippets



These code blocks show a production-conscious inference design: consistent features, version control, vectorized scoring, and operational logging.

Loads approved production model from registry (models/registry.json), not newest .pkl file. Prevents serving unapproved candidate models.

```
model, meta = get_production_model()

if "feature_cols" in meta:
    trained_cols = meta["feature_cols"]
    X_df = pd.DataFrame(X, columns=feature_cols)
    for c in trained_cols:
        if c not in X_df.columns:
            X_df[c] = 0.0
    X = X_df[trained_cols].values.astype(np.float32)
```

Guarantees exact training-time feature order/structure. Handles new OHE categories during inference by zero-filling missing columns.

Vectorized batch scoring — processes entire CSV/API batch in single call. LightGBM auto-parallelizes across CPU cores.

```
preds = model.predict(X)
probs = model.predict_proba(X)[:, 1]
```

Extracts purchase probability ($P(\text{Revenue}=1)$) for ranking/thresholding. Business uses $\text{prob} > 0.55$ to trigger offers.

```
df["intent_level"] = pd.cut(
    probs,
    bins=[0, 0.25, 0.55, 1.01],
    labels=["low", "medium", "high"],
    right=False,
)
```

Online Serving with FastAPI



`serve.py` provides real-time scoring for one session at a time through a REST API. The file explains that the production model should be loaded once at startup to avoid per-request disk I/O overhead.

Techniques used

- FastAPI for serving.
- Pydantic schemas for request and response validation.
- Production-model loading from the registry.
- Shared feature building with training-time medians.
- Probability-to-intent mapping for readable business output.
- Logging of both predictions and latency.

Startup syntax

Loads production model once at server startup (not per-request). Saves 50-500ms I/O latency per prediction. Uses registry → only approved models served.

```
@app.on_event("startup")
def load_model_on_startup():
    global _model, _meta
    create_directories()
    _model, _meta = get_production_model()
```

Request schema syntax

Automatic input validation + documentation. Rejects invalid data (negative BounceRates, missing fields). Generates Swagger UI at /docs.

```
class SessionRecord(BaseModel):
    Administrative: float = Field(..., ge=0, example=2)
    Administrative_Duration: float = Field(..., ge=0, example=80.0)
```



```
ProductRelated: float = Field(..., ge=0, example=15)
BounceRates: float = Field(..., ge=0, le=1, example=0.02)
ExitRates: float = Field(..., ge=0, le=1, example=0.04)
PageValues: float = Field(..., ge=0, example=12.5)
Month: str = Field(..., example="Nov")
VisitorType: str = Field(..., example="Returning_Visitor")
Weekend: bool = Field(..., example=False)
```

Prediction endpoint syntax

Single-session scoring: JSON → DataFrame → training=False features → prediction. Same feature pipeline as batch_infer.py.

```
@app.post("/predict", response_model=PredictionResponse, tags=["Inference"])
def predict_purchase(record: SessionRecord):
    row_df = pd.DataFrame([record.dict()])
    row_df["Weekend"] = row_df["Weekend"].astype(str)
    X, _, feature_cols = build_features(row_df, training=False)
```

Probability banding syntax

Probability → Actionable decision: Maps raw ML output to business rules. Returns both probability (precise) + intent_level (actionable).

```
if prob < 0.25:
    intent_level = "low"
elif prob < 0.55:
    intent_level = "medium"
else:
    intent_level = "high"
```

This design turns a raw probability into a simpler business-facing label while preserving the exact score as well.

Drift Detection and Retraining Logic



`retrain.py` handles the post-deployment phase after drift is discovered. Its docstring clearly explains the new setting: it is now late November or early December, new data has accumulated, and the model may no longer reflect current behavior.

Drift signals described in the file

- `PageValues` shifted upward.
- `ProductRelated` browsing depth increased.
- `BounceRates` changed.
- `ExitRates` changed.
- `SpecialDay` behavior became active in the holiday context.

Four retraining trigger rules

Rule	Condition	Purpose
Rule 1	Any high-importance feature drifts	Immediate warning when dominant features shif.
Rule 2	At least 2 features drift	Reduces false alarms from one noisy feature.
Rule 3	F1 drops more than 5 percentage points	Captures concept drift even if input drift is less obvious.
Rule 4	Minimum 14-day interval between retrains	Prevents overreacting to short-term noise.

Rule-evaluation Code Snippets

Checks if experiment log exists. Returns "no retrain needed" if missing to safely skip degradation analysis.

```
def _check_f1_degradation() -> tuple:
    if not os.path.exists(EXPERIMENT_LOG):
        return False, 0.0, None, None
```

Triggers retrain if F1 dropped >5pp from previous run. Returns decision + metrics for logging



```
return delta > F1_DEGRADATION_THRESHOLD, round(delta, 4), round(current_f1, 4), round(baseline_f1, 4)
```

Three retraining strategies

1. **Finetune:** warm-start LightGBM on new holiday data only.
2. **Mixed retrain:** combine sampled original data with all new data and train a fresh model.
3. **Reexperiment:** search a small hyperparameter grid on the combined data and rerun the pipeline with the best configuration.

Fine-tuning syntax

This trains the finetuned LightGBM model on `X_ft_train` and `y_ft_train` while **starting from the existing production model** through `init_model=existing_model.booster_`. In this code, the new model does not start from scratch; it continues training using the already learned booster state.

```
finetuned.fit(X_ft_train, y_ft_train, init_model=existing_model.booster_)
```

Mixed retrain syntax

This creates one combined dataset by joining the sampled old training data with the current data into a single DataFrame. The second line sends that combined dataset into `run_pipeline()` through `extra_train_df`, so the full pipeline is rerun using this expanded training input.

```
combined_df = pd.concat([sampled_df, current_df], ignore_index=True)
metrics = run_pipeline(extra_train_df=combined_df)
```

Reexperiment syntax

This defines a list of hyperparameter combinations to test during the reexperiment strategy. Each dictionary represents one candidate LightGBM configuration, and later the script loops through this grid to compare model performance.

```
grid = [
    {"n_estimators": 400, "learning_rate": 0.05, "num_leaves": 31},
    {"n_estimators": 500, "learning_rate": 0.05, "num_leaves": 63},
    {"n_estimators": 500, "learning_rate": 0.03, "num_leaves": 63},
```

© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.



```
{"n_estimators": 600, "learning_rate": 0.03, "num_leaves": 127},  
]
```

The file recommends the mixed strategy for seasonal drift because it balances new seasonal behavior with old non-holiday behavior and avoids catastrophic forgetting.

Logging, Monitoring, and Observability

The notebook repeatedly treats observability as a first-class requirement. `plot_logs.py` reads experiment, drift, latency, and registry outputs and generates an HTML report for inspection.

Operational artifacts

- `logs/experiments.csv` for run history.
- `logs/drift.csv` for drift detection results.
- `logs/latency.csv` for inference-time monitoring.
- `logs/lineage.json` for traceability across code, data, and model state.
- `logs/retrain.csv` for retraining history.
- `models/registry.json` for lifecycle state.

Why this is important

These artifacts convert a model into an observable system rather than an opaque script. The walkthrough explicitly says that each stage produces a concrete artifact and that this is how ML systems become traceable and manageable in production.

Conceptual Flow of the Whole Project



The project should be understood as a story, not just a folder of scripts. The story begins with a plausible but unsafe baseline, improves into a controlled model-comparison process, then grows into a full production pipeline with validation, feature persistence, gated deployment, registry-based promotion, drift monitoring, retraining, batch scoring, API serving, and log visualization.

A useful classroom explanation is this: `baseline.py` shows why notebook success is not production success, `model_selection.py` shows how to choose the right model properly, `main.py` operationalizes that choice, `retrain.py` responds to change, `batch_infer.py` and `serve.py` expose predictions, and `plot_logs.py` helps humans see what the system is doing.

Important Code Patterns to Highlight

1. Entry-point orchestration

This block runs `run_pipeline()` only when `main.py` is executed directly as a script. It makes `main.py` the top-level entry point for the full pipeline instead of executing the pipeline automatically when the file is imported elsewhere.

```
if __name__ == "__main__":  
    run_pipeline()
```

2. Stratified evaluation split

This splits the dataset into train and test portions, with 20% reserved for testing and a fixed seed for reproducibility. The `stratify=y_all` part preserves the same class ratio in both splits so the minority purchase class does not get distorted.

```
train_test_split(X_all, y_all, test_size=0.20, random_state=SEED, stratify=y_all)
```



3. Imbalance-aware training

This calculates the ratio of negative samples to positive samples in the training set. That value is later used to give more importance to the minority positive class during model training

```
scale_pos_weight = (y_train == 0).sum() / max(pos_train, 1)
```

4. Consistent inference pipeline

This builds features for inference using the previously saved training-time transformations instead of fitting new ones on the current data. It keeps inference consistent with training so the model receives the same feature structure it was originally trained on.

```
build_features(df, training=False)
```

5. Production-model loading

This loads the currently approved production model together with its metadata from the registry. The metadata is then used for things such as model version tracking and trained feature-column alignment

```
model, meta = get_production_model()
```

6. Warm-start retraining

This fits the fine-tuned model on new training data while initializing it from the existing production booster. In code terms, the model continues from the earlier learned state instead of starting as a brand-new model from scratch

```
finetuned.fit(X_ft_train, y_ft_train, init_model=existing_model.booster_)
```

7. Batch vectorized prediction

The first line generates predicted class labels for all rows in X, and the second line extracts the probability of the positive purchase class for all rows. Because the whole feature matrix is passed at once, the script performs batch scoring in a vectorized way rather than scoring row by row



```
preds = model.predict(X)
probs = model.predict_proba(X)[:, 1]
```

8. API schema enforcement

This class defines part of the expected structure for incoming API request data. It ensures that fields such as Month, VisitorType, and Weekend are validated against the declared data types before prediction logic is applied

```
class SessionRecord(BaseModel):
    Month: str
    VisitorType: str
    Weekend: bool
```

Conclusion

This project implements a **complete end-to-end MLOps pipeline** for predicting e-commerce purchases on an imbalanced dataset, moving beyond just model building to a production-ready system. It begins with a baseline, improves model selection using proper metrics, and then integrates the chosen model into a structured pipeline that handles data validation, feature engineering, training, evaluation, and controlled deployment.

The system further extends into real-world operations by incorporating **model registry, monitoring, drift detection, retraining strategies, batch and real-time inference, and logging for observability**. Overall, the project demonstrates how a machine learning model is not just built, but **operationalized, monitored, and continuously improved in a production environment**.

DISCLAIMER: This learning course may contain images, both still and moving, including photographs, advertisements, third-party trademarks, video clips etc. that may constitute copyrighted material procured from open-source, public domain, social media, or other print or digital publications, with the sole and bona fide intention of imparting education, disseminating information and illustrating an academic concept or an issue. We believe that this constitutes a 'fair dealing' exception under Section 52 of the Indian Copyright Act, 1957. Throughout this course, you may see examples of different brands and
© Copyright. upGrad Education Pvt. Ltd. All Rights Reserved.



institutions. These have been chosen from specific cases in a way that would aid in demonstrating concepts. These do not reflect the current performance of any company. Any opinion expressed is strictly for educational purposes.